

C  puto Concurrente 2026

Pr  ctica 2

Exclusi  n Mutua: Locks y Pools en Java

Profa: Gilde Valeria Rodr  guez

1 Contexto

En el mundo de la programaci  n concurrente surgen problemas cuando m  ltiples hilos tratan de modificar un recurso compartido. Un ejemplo claro de esto es el ejemplo del *ContadorNaive* en la pr  ctica anterior, existen errores e inconsistencias en los resultados.

A continuaci  n se definen dos t  rminos para referirnos a este tipo de situaciones en el mundo concurrente:

- Condici  n de carrera (*race condition*): Ocurre cuando en una ejecuci  n dos o m  s hilos acceden a un recurso compartido (por ejemplo, una variable) al mismo tiempo, y el resultado depende del orden de ejecuci  n [OW04].
- Carrera por los datos (*data race*): Ocurre cuando dos o m  s hilos realizan una llamada a un m  todo de un objeto compartido al mismo tiempo, y al menos una llamada es una escritura [MLS24].

Por ejemplo, en el programa del *ContadorNaive* existe una *race condition* porque la variable *counter* puede ser accedida por varios hilos al mismo tiempo y el resultado de *counter* puede variar en distintas ejecuciones dependiendo del orden en el que se ejecutan los hilos. Tambi  n existe una *data race* porque la variable *counter* puede ser le  da y modificada con un m  todo de escritura por al menos dos hilos al mismo tiempo.

El concepto de **secci  n cr  tica** nos permite evitar *data races* y *race conditions*. Una secci  n cr  tica es un bloque de c  digo que solo puede ser ejecutado por un hilo a la vez.

Java provee varias implementaciones de secciones cr  ticas, en esta pr  ctica abordaremos:

-    la palabra reservada *synchronized* y
-    la interface *Lock*.

En la clase te  rica vimos que un objeto Candado es un objeto que resuelve el problema de exclusi  n mutua al permitir que solo un hilo lo obtenga a la vez. La palabra reservada **synchronized** funciona de forma similar a un candado, la diferencia es que no solo previene que dos o m  s hilos accedan a un bloque de c  digo, adem  s previene que dos o m  s hilos accedan a distintos m  todos que modifican un mismo objeto al mismo tiempo. Cada objeto en Java tiene asociado un candado, es por eso que **synchronized** permite crear secciones cr  ticas por objeto [OW04].

La clase **Lock** (*java.util.concurrent.locks*) nos permite crear secciones cr  ticas de manera m  s flexible que al utilizar *synchronized*. La raz  n es que el *scope* (*alcance*) de *synchronized* comprende todos los m  todos en los cuales se utiliz   *synchronized*, en cambio, el *scope* de un objeto *Lock* solo comprende el bloque de c  digo que se encuentra entre la llamada *lock()* (adquirir el candado) y *unlock()* (dejar el candado). La clase incluye otros m  todos y existen varios tipos de objetos de tipo *Lock*: <https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/locks/package-summary.html>.

La clase **Semaphore** es b  sicamente una generalizaci  n de un candado que incluye un contador. Es una generalizaci  n de un candado porque permite que un n  mero n ($n \geq 1$) de hilos accedan

a una sección crítica <https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Semaphore.html>.

1.1 Pools de hilos

A partir de Java (J2SE 5.0) se incorporaron las implementaciones de *pools de hilos*. El objetivo de una *pool* es crear un grupo de hilos en espera de tareas por ejecutar, una vez que existe una tarea, uno de los hilos la toma, la ejecuta, finaliza y vuelve a esperar por otra. Utilizamos *pools* para reusar hilos (mejorar el rendimiento) y mejorar el diseño del programa. No conviene utilizar *pools* si realizamos procesamiento secuencial (en *batch*) o si no importa la cantidad de hilos que ocupemos en un programa.

Para implementar una *pool* en Java utilizamos la interfaz *Executor* <https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Executor.html>, la cual tiene un método *execute()* que recibe como parámetro un objeto *Runnable*.

```
1 package java.util.concurrent;
2     public interface Executor {
3         public void execute(Runnable task);
4     }
```

También provee una subinterfaz de *Executor*, llamada *ExecutorService* que implementa los siguientes métodos:

```
5 package java.util.concurrent;
6     public interface ExecutorService extends Executor {
7         void shutdown( ); //termina las tareas que se hayan enviado al ejecutor, pero
            no realiza nuevas
8         List shutdownNow( ); //trata de terminar todas las tareas, incluso las que se
            han enviado y regresa una lista de las que no se empezaron
9         boolean isShutdown( );
10        boolean isTerminated( ); //es verdadero si el estado de la pool esta en "
            terminado"
11        boolean awaitTermination(long timeout, TimeUnit unit)
            throws InterruptedException; //Podemos esperar a que termine una
            determinado intervalo de tiempo
12        // Si mandamos una tarea via "submit" nos devuelven un objeto Future, nos sirve
            para checar si la tarea ha terminado
13        <T> Future<T> submit(Callable<T> task);
14        <T> Future<T> submit(Runnable task, T result);
15        Future<?> submit(Runnable task);
16        <T> List<Future<T>> invokeAll(Collection<Callable<T>> tasks)
            throws InterruptedException;
17        <T> List<Future<T>> invokeAll(Collection<Callable<T>> tasks,
            long timeout, TimeUnit unit)
            throws InterruptedException; //Ejecuta todos los metodos en una
            coleccion
18        <T> T invokeAny(Collection<Callable<T>> tasks)
            throws InterruptedException, ExecutionException; //ejecuta alguno y
            termina
19        <T> T invokeAny(Collection<Callable<T>> tasks, long timeout, TimeUnit unit)
            throws InterruptedException, ExecutionException, TimeoutException;
20    }
```

La clase *Future* se ve de la siguiente forma:

```
28 public interface Future<V> {
29     V get( ) throws InterruptedException, ExecutionException; //Devuelve el
        resultado del callable, espera de ser necesario
30     V get(long timeout, TimeUnit unit)
```

```

31         throws InterruptedException, ExecutionException, TimeoutException; //Se
           bloquea hasta que se devuelva el resultado o hasta que expire el
           timeout
32     boolean isDone( ); //Devuelve verdadero si la tarea ya se realizo
33     boolean cancel(boolean mayInterruptIfRunning);
34     boolean isCancelled( ); //
35 }

```

Si el método *cancel()* se llama, el objeto Callable, puede nunca empezar, pudo haber terminado y el método ya no tiene efecto o puede ser que se esté ejecutando y el hilo sea interrumpido aunque el objeto no cese por completo.

Java también tiene implementado una clase (*ThreadPoolExecutor*), la cual implementa la interface *ExecutorService*. La cual especifica el cómo ejecutar y terminar las tareas, permite especificar las características de la cola para las tareas. Sin embargo, lo abordaremos más adelante.

2 Ejemplos

En el siguiente link: https://github.com/surindt/FC_CConcurrente/tree/main/Programas_P2

- Programa 1 (SynchronizedExample1): Utilización de synchronized, el método run es una sección critica, cada hilo lo ejecuta, y hasta que no termina de ejecutarlo lo deja.
- Programa 2 (SynchronizedExample2): Utilización de synchronized, ejemplo de un problema con el alcance
- Programa 3 (LockExample1): Ejemplo de un contador que utiliza un candado predefinido de Java
- Programa 4 (ExampleExecutor): Ejemplo de una pool de hilos que ejecuta n tareas (runnables)
- Programa 5 (CounterPool): Una pool de hilos que ejecuta un contador
- Programa 6 (CounterPoolCallable): Una pool de hilos que ejecuta un contador, utiliza futures y callables
- Programa 7 (ColaSecuencial): Implementación de una cola secuencial, utiliza la clase Nodo
- Programa 8 (Scheduler): Programa para completar, utiliza la clase Tarea

3 Ejercicios

Instrucciones:

- Entrega en un **PDF** las respuestas de los ejercicios que no requieran implementarse, en los ejercicios que requieran implementación añade una breve descripción de los programas que entregas (sus nombres y qué hacen).
- Solo un integrante del equipo debe subir la práctica. Los demás integrantes deben marcar la tarea como entregada y escribir, en un comentario privado dentro de la práctica, el nombre completo de la persona que realizó la entrega. Todos los programas y el PDF deben ir dentro de un **archivo ZIP**. El archivo ZIP debe seguir el siguiente formato de nombre: P#k_ApellidoPaternoNombre, usando el apellido y nombre de la persona que entrega la práctica.
- Cada ejercicio debe tener un programa que indique “Implementar” si aplica.
- Recuerda que debes utilizar **Java 21 LTS**.
- **Si no compila utilizando Java 21 LTS o si no se entrega la breve descripción de los programas se penalizará.**

Tiempo de elaboración: ≈ 2.5hr

Total de puntos: 100

Nota: En esta práctica **no** es necesario utilizar otros objetos de la biblioteca Concurrent de java que no hayan sido mencionados en el Contexto 1 de esta práctica, por ejemplo: *getAndIncrement*, *ArrayBlockingQueue*, *ConcurrentLinkedQueue*. Muchos de estos objetos ya predefinidos en *java* son implementaciones *linealizables*: con consistencia, y en esta práctica no los necesitamos.

1. El programa *ColaSecuencial* es una implementación de una cola secuencial. Implementa una Cola concurrente utilizando una pool de hilos (*ExecutorService*). No utilices candados ni *synchronized*.
2. Ejecuta varias veces tu implementación con diferentes secuencias de llamadas a métodos ¿Tu implementación funciona de acuerdo a lo que se espera de una cola o suceden *inconsistencias*? Por ejemplo,
 - 1) Se hace *enq(a)*, *enq(b)* y *deq()* y al final ambos elementos *a* y *b* están en la cola.
 - 2) Se realiza un *enq(a)* y un *enq(b)* y la cola solo contiene al elemento *a* o *b*.
 - 3) Se desencola dos veces el mismo elemento.

Si existe una ejecución así toma una captura pantalla del resultado de tu programa que lo ejemplifique.

Hint: Para analizar si hay inconsistencias, utiliza la interfaz Future, solo ten cuidado con el método get() ya que fuerza a que el resultado del future sea devuelto en ese momento.

Importante: El orden en el que se suben las tareas con submit y se guardan en los futures, no es el orden en el que se realizan, solo es el orden en el que se les asignan a los hilos. Nuestro sistema es *asíncrono*, es decir, cada hilo puede tardarse más o menos en ejecutar la tarea que se le asignó.

3. ¿Existen *data races* o *race-conditions*? Explica en qué variables y por qué suceden.
4. Utiliza candados y/o *synchronized* en tu implementación de forma que los métodos *enq()* y *deq()* sean una sola sección crítica y contesta: ¿existen inconsistencias?

Importante: El orden en el que se suben las tareas con submit y se guardan en los futures, no es el orden en el que se realizan, solo es el orden en el que se les asignan a los hilos. Nuestro sistema es *asíncrono*, es decir, cada hilo puede tardarse más o menos en ejecutar la tarea que se le asignó.

5. En una pool de hilos (*ExecutorService*), ¿si utilizamos más hilos equivale a un mayor *throughput*? Argumenta porqué.

6. **Problema.** Supón que perteneces a un equipo de trabajo en el cual estás a cargo de dar acceso a un servidor. Seis personas te pueden mandar tareas para que las ejecutes en el servidor. Las tareas del hilo 0 y 2 tardan $500ns$, las del hilo 1 tardan $2000ns$ y las de los hilos 3, 4 y 5 tardan $3000ns$.

Tienes la instrucción de no dar acceso a más de tres al mismo tiempo y de no aceptar al mismo tiempo las tareas del hilo 0 y 2. Diseña e implementa una solución. *Hint: Apóyate del programa `Scheduler` y `Tarea`, el semáforo inicializado en 3 representa que solo pueden entrar 3 al mismo tiempo, decide como utilizarlo en el programa **Tarea** (colocando sus métodos *acquire* y *release*); además, decide como utilizar un candado, para restringir aceptar las tareas de los hilos 0 y 2 al mismo tiempo.*

- ¿Tu implementación cumple con Justicia?
- Sino es así, describe cómo podrías garantizarla.

References

- [MLS24] Trevor Brown Michael L. Scott. *Shared-Memory Synchronization*. Springer Cham, 30 January 2024.
- [OW04] S. Oaks and H. Wong. *Java Threads: Understanding and Mastering Concurrent Programming*. O'Reilly Media, 2004.